

CIC0198 - TÉCNICAS DE PROGRAMAÇÃO 2

Princípios de Projeto Modular

Daniel Porto – daniel.porto@unb.br

Departamento de Ciência da Computação – CIC

Universidade de Brasília – UnB

Material adaptado do Prof. Marco Tulio Valente



UnB | IE | CIC

Princípios de Projeto Modular

Introdução

Propriedades de Projeto

Princípios de Projeto

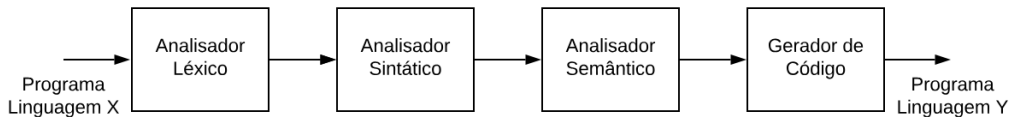
COMPILADOR

Como implementar um compilador?



Compilador

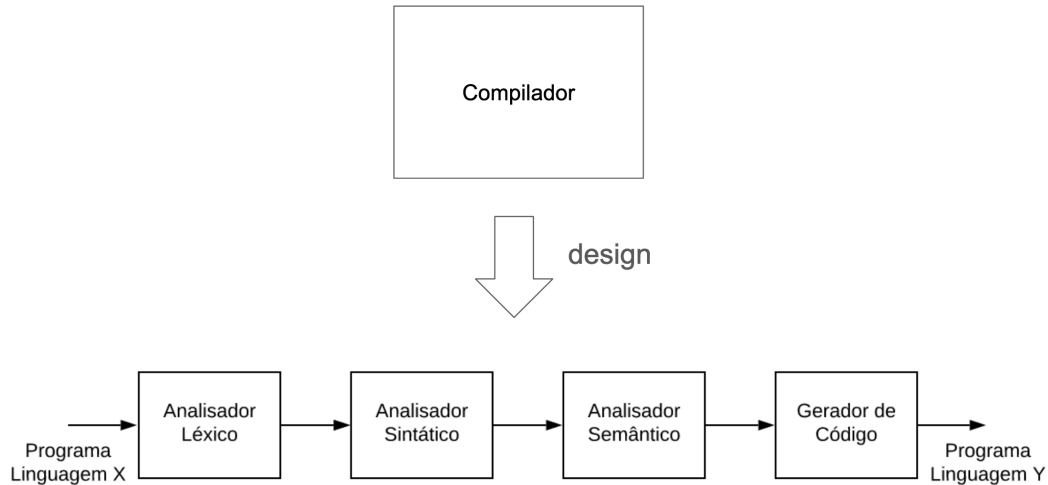
COMPILADOR



O QUE É DESIGN DE SOFTWARE?



O QUE É DESIGN DE SOFTWARE?



PROJETO (DESIGN) DE SOFTWARE

Dividir para conquistar

Quebrar um “problema grande” em partes menores

MÓDULOS

Nome que se dá em design para tais “partes menores”

Dependendo do contexto, um módulo pode ser:

- ▶ função
- ▶ classe
- ▶ pacote
- ▶ componente
- ▶ biblioteca
- ▶ microsserviço
- ▶ etc

Módulo: interface + implementação

O QUE VAMOS ESTUDAR?

Propriedades de “bons projetos” de software

- ▶ Integridade Conceitual
- ▶ Ocultamento de Informação
- ▶ Coesão
- ▶ Acoplamento

PRINCÍPIOS DE PROJETO

- ▶ Responsabilidade Única
- ▶ Segregação de Interfaces
- ▶ Prefira Interfaces a Classes
- ▶ Aberto/Fechado
- ▶ Demeter
- ▶ Substituição de Liskov

Princípios de Projeto Modular

Introdução

Propriedades de Projeto

Princípios de Projeto

INTEGRIDADE CONCEITUAL



INTEGRIDADE CONCEITUAL EM 1 SLIDE

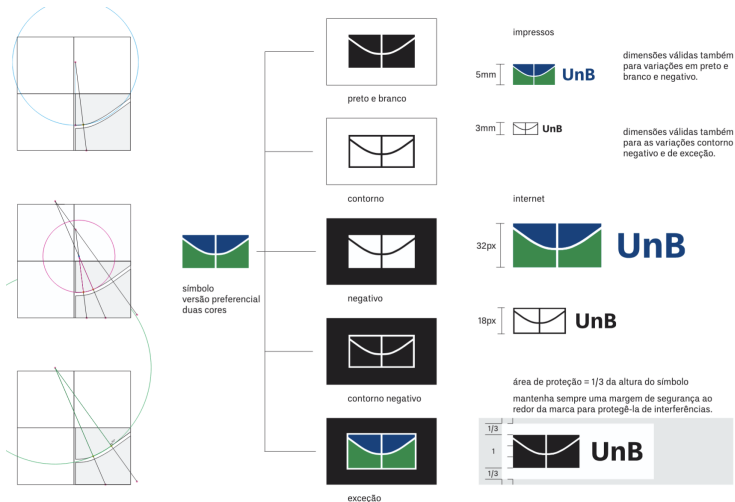


Exemplo



Contra-Exemplo

INTEGRIDADE CONCEITUAL DA UNB



INTEGRIDADE CONCEITUAL DA UNB



[8.8]



[8.9]



[8.10]



[8.11]



[8.12]



[8.13]



[8.14]



[8.15]



INTEGRIDADE CONCEITUAL VALE PARA:

- ▶ Funcionalidades
- ▶ Interface com usuário
- ▶ Projeto
- ▶ Implementação
- ▶ etc

EXEMPLO (INTERFACE COM O USUÁRIO)

Botão “sair” é idêntico em todas as telas

Se um sistema usa tabelas para apresentar resultados, todas as tabelas têm o mesmo leiaute

Todos os resultados são mostrados com 2 casas decimais

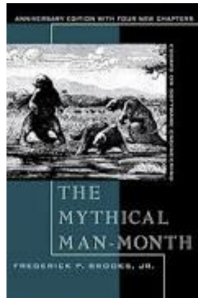
EXEMPLOS (PROJETO/CÓDIGO)

Todas as variáveis seguem o mesmo padrão de nomes – contra-exemplo:
`nota_total` vs `notaMedia`

Todas as páginas usam o mesmo framework (mesma versão)

Se um problema é resolvido com uma estrutura de dados X, todos os problemas parecidos também usam X

Integridade conceitual é a consideração mais importante no projeto de sistemas – Fred Brooks



INTEGRIDADE CONCEITUAL

Motivo:



INTEGRIDADE CONCEITUAL

Motivo: integridade conceitual facilita uso e entendimento de um sistema

OCULTAMENTO DE INFORMAÇÃO (INFORMATION HIDING)



ORIGEM DO CONCEITO (DAVID PARNAS, 1972)

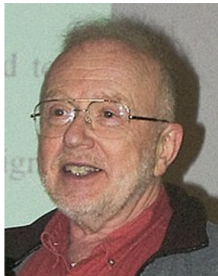
On the Criteria To Be Used in Decomposing Systems into Modules

D.L. Parnas
Carnegie-Mellon University

This paper discusses modularization as a mechanism for improving the flexibility and comprehensibility of a system while allowing the shortening of its development time. The effectiveness of a “modularization” is dependent upon the criteria used in dividing the system into modules. A system design problem is presented and

Introduction

A lucid statement of the philosophy of modular programming can be found in a 1970 textbook on the design of system programs by Gouthier and Pont [1, ¶10.23], which we quote below:¹



OCULTAMENTO DE INFORMAÇÃO

```
1 import java.util.Hashtable;
2
3 public class Estacionamento {
4
5     public Hashtable<String, String> veiculos;
6
7     public Estacionamento() {
8         veiculos = new Hashtable<String, String>();
9     }
10
11     public static void main(String[] args) {
12         Estacionamento e = new Estacionamento();
13         e.veiculos.put("TCP-7030", "Uno");
14         e.veiculos.put("BNF-4501", "Chevette");
15         e.veiculos.put("JKL-3481", "Opala");
16     }
17 }
```

Qual é o problema aqui?

OCULTAMENTO DE INFORMAÇÃO

```
1 import java.util.Hashtable;
2
3 public class Estacionamento {
4
5     public Hashtable<String, String> veiculos;
6
7     public Estacionamento() {
8         veiculos = new Hashtable<String, String>();
9     }
10
11     public static void main(String[] args) {
12         Estacionamento e = new Estacionamento();
13         e.veiculos.put("TCP-7030", "Uno");
14         e.veiculos.put("BNF-4501", "Chevette");
15         e.veiculos.put("JKL-3481", "Opala");
16     }
17 }
```

Qual é o problema aqui?

Problema: clientes precisam manipular uma estrutura de dados interna da classe, para estacionar um veículo

PROBLEMA

Classes precisam de um pouco de “privacidade”

Até para evoluir de forma independente dos clientes

Código anterior: clientes manipulam a hashtable

Comparação: clientes não podem entrar na cabine do estacionamento e eles mesmo anotar os dados do seu carro no “livro” do estacionamento

UMA VERSÃO MELHOR

```
1 import java.util.Hashtable;
2 public class Estacionamento {
3     private Hashtable<String, String> veiculos;
4
5     public Estacionamento() {
6         veiculos = new Hashtable<String, String>();
7     }
8     public void estaciona(String placa, String veiculo) {
9         veiculos.put(placa, veiculo);
10    }
11    public static void main(String[] args) {
12        Estacionamento e = new Estacionamento();
13        e.estaciona("TCP-7030", "Uno");
14        e.estaciona("BNF-4501", "Gol");
15        e.estaciona("JKL-3481", "Corsa");
16    }
17 }
```

OCULTAMENTO DE INFORMAÇÃO

Classes devem ocultar detalhes internos de sua implementação (usando modificador `private`)

Principalmente aqueles sujeitos a mudanças

Adicionalmente, interface da classe deve ser estável

Interface = conjunto de métodos públicos de uma classe

SIGNIFICADOS DA PALAVRA INTERFACE

Interface: conjunto de métodos públicos de uma classe

Interface: Em uma linguagem (palavra reservada)

SIGNIFICADOS DA PALAVRA INTERFACE

Interface: conjunto de métodos públicos de uma classe

Interface: Em uma linguagem (palavra reservada)

Existe ainda um terceiro significado:

Interface com o usuário (UI), interface gráfica com o usuário (GUI), interface mobile, etc Fora do escopo deste curso

INTERFACE EM KOTLIN

```
1 interface Animal {  
2     fun makeSound()  
3 }  
4  
5 class Dog : Animal {  
6     override fun makeSound() {  
7         println("Woof!")  
8     }  
9 }  
10  
11 class Cat : Animal {  
12     override fun makeSound() {  
13         println("Meow!")  
14     }  
15 }
```

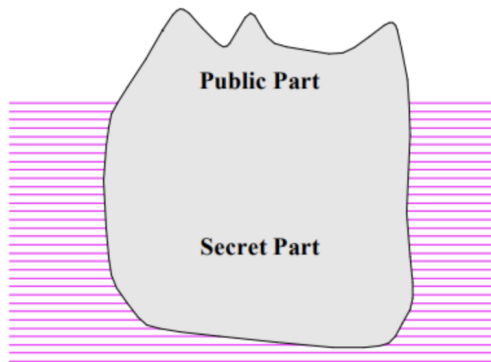
```
16  
17 class MyClass {  
18     fun f(animal: Animal) {  
19         // ...  
20         animal.makeSound()  
21         // ...  
22     }  
23 }
```

INTERFACE

Mesmo que uma classe não implemente uma interface (palavra reservada), ela possui uma interface (seus métodos públicos)

BONS MÓDULOS

Bom módulos são semelhantes a icebergs: pequena parte pública e visível;
grande parte submersa e privativa

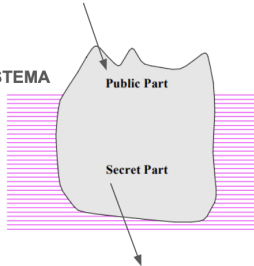


Fonte: Bertrand Meyer, Object-oriented software construction, 1997

GENERALIZANDO...

API (REST, gRPC, GraphQL)

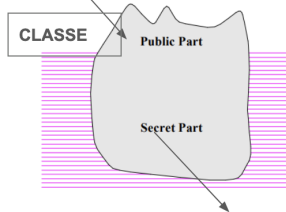
SISTEMA



Sistema completo

Assinatura dos métodos públicos

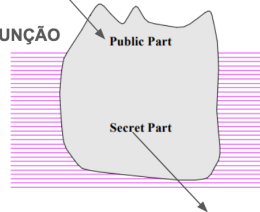
CLASSE



Implementação de métodos públicos; métodos privados

Assinatura de uma função ou método

FUNÇÃO



Implementação de uma função ou método

ISSO TEM OUTRO NOME

Outro nome:

ISSO TEM OUTRO NOME

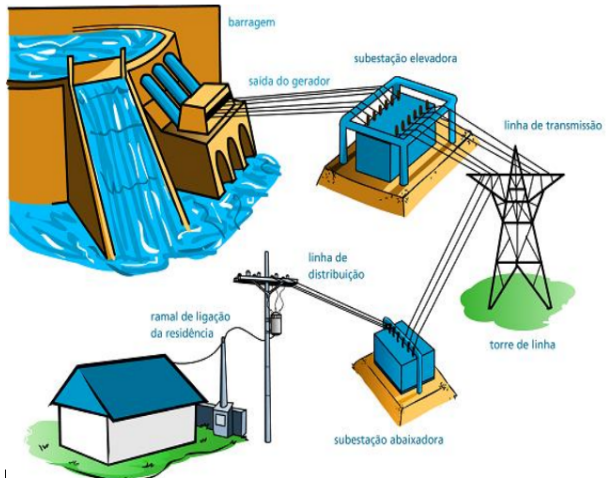
Outro nome: **encapsulamento**

ISSO TEM OUTRO NOME

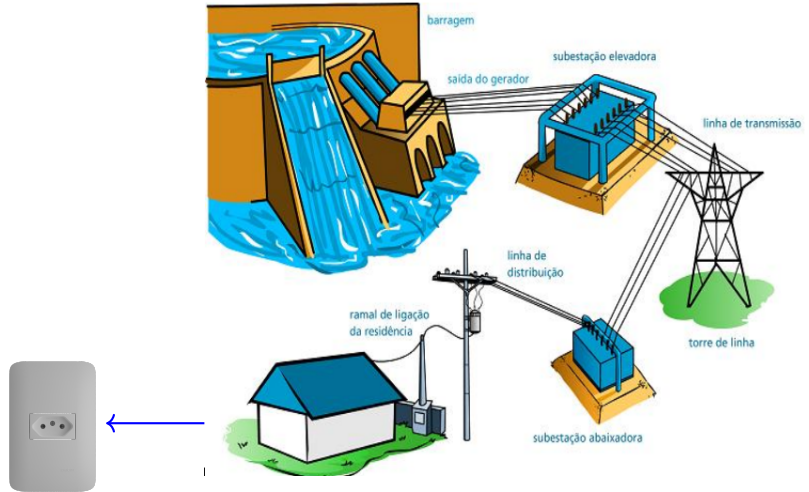
Outro nome: **encapsulamento**

Alguns autores, preferem o nome encapsulamento, mas com o mesmo significado de ocultamento de informação

INFORMATION HIDING EM 1 SLIDE



INFORMATION HIDING EM 1 SLIDE



Uma classe deve ter uma única função, isto é, oferecer um único serviço
Vale também para outras unidades: funções, métodos, pacotes, etc.

COESÃO - CONTRA-EXEMPLOS

```
1 import math
2
3 def sen_or_cos(x, func_type):
4     if func_type.lower() == '1':
5         return math.sin(x)
6     else
7         return math.cos(x)
```

COESÃO - CONTRA-EXEMPLOS

```
1 import math
2
3 def sen_or_cos(x, func_type):
4     if func_type.lower() == '1':
5         return math.sin(x)
6     else
7         return math.cos(x)
```

```
1 #include <string>
2 using namespace std;
3
4 class Estacionamento {
5 public:
6     int vagas_totais;
7     int vagas_ocupadas;
8     //...
9     string nome_gerente;
10    string fone_gerente;
11    string cpf_gerente;
12    string endereco_gerente;
13 }
```

COESÃO - EXEMPLO

```
1 public class Pilha<T> {  
2     public Pilha() {...}  
3  
4     public void push(T elemento) {...}  
5  
6     public T pop() {...}  
7  
8     public T peek() {...}  
9  
10    public boolean isEmpty() {...}  
11  
12    public int size() {...}  
13  
14    public void clear() {...}  
15 }
```

ACOPLAMENTO

Nenhuma classe é uma ilha ...

Classes dependem umas das outras (chamam métodos de outras classes, estendem outras classes, etc)

ACOPLAMENTO

Nenhuma classe é uma ilha ...

Classes dependem umas das outras (chamam métodos de outras classes, estendem outras classes, etc)

A questão principal é a qualidade desse acoplamento

Dois tipos:

- ▶ Acoplamento aceitável ("bom")
- ▶ Acoplamento ruim

ACOPLAMENTO ACEITÁVEL

Classe A usa uma classe B e:

- ▶ B oferece um serviço útil para A
- ▶ B possui uma interface estável
- ▶ A somente chama métodos da interface de B

ACOPLAMENTO ACEITÁVEL

```
1 import java.util.Hashtable;◀◀
2 public class Estacionamento {
3     private Hashtable<String, String> veiculos;◀◀
4
5     public Estacionamento() {
6         veiculos = new Hashtable<String, String>();
7     }
8     public void estaciona(String placa, String veiculo) {
9         veiculos.put(placa, veiculo);
10    }
11    public static void main(String[] args) {
12        Estacionamento e = new Estacionamento();
13        e.estaciona("TCP-7030", "Uno");
14        e.estaciona("BNF-4501", "Gol");
15        e.estaciona("JKL-3481", "Corsa");
16    }
17 }
```

ACOPLAMENTO RUIM

Classe A usa uma classe B:

- ▶ Mas interface da classe B é instável
- ▶ Ou então o uso não ocorre via interface de B

ACOPLAMENTO RUIM

Classe A usa uma classe B:

- ▶ Mas interface da classe B é instável
- ▶ Ou então o uso não ocorre via interface de B

**Como uma classe A estar acoplada a uma classe B
sem ser via a interface de B?**

ACOPLAMENTO RUIM

```
1 class B {  
2     private void g() {  
3         int total;  
4         // computa valor de total  
5         File f = File.open("arq1.db");  
6         f.writeInt(total);  
7         ...  
8         f.close();  
9     }  
10 }
```

ACOPLAMENTO RUIM

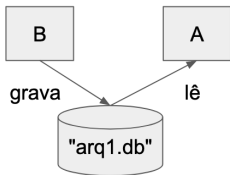
```
1 class B {  
2     private void g() {  
3         int total;  
4         // computa valor de total  
5         File f = File.open("arq1.db");  
6         f.writeInt(total);  
7         ...  
8         f.close();  
9     }  
10 }
```

```
1 class A {  
2     private void f() {  
3         int total;  
4         ...  
5         File f = File.open("arq1.db");  
6         total = f.readInt();  
7         ...  
8     }  
9 }
```

ACOPLAMENTO RUIM

```
1 class B {  
2     private void g() {  
3         int total;  
4         // computa valor de total  
5         File f = File.open("arq1.db");  
6         f.writeInt(total);  
7         ...  
8         f.close();  
9     }  
10 }
```

```
1 class A {  
2     private void f() {  
3         int total;  
4         ...  
5         File f = File.open("arq1.db");  
6         total = f.readInt();  
7         ...  
8     }  
9 }
```

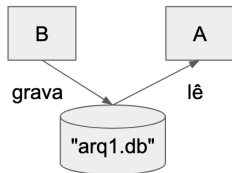


ACOPLAMENTO RUIM

Mudanças internas em B podem impactar A

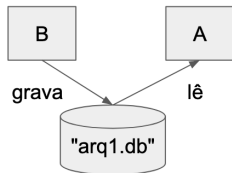
Exemplo: B pode mudar o formato do arquivo ou mesmo deixar de salvar o dado que é lido por A

Na literatura, esse tipo de acoplamento é chamado também de acoplamento evolutivo ou lógico



Como resolver esse problema?

Como tornar um acoplamento ruim em bom?



TORNANDO ACOPLAMENTO RUIM EM BOM

```
1 class B {  
2     int total;  
3  
4     public int getTotal() {  
5         return total;  
6     }  
7  
8     private void g() {  
9         // computa valor de total  
10        File f = File.open("arq1");  
11        f.writeInt(total);  
12        ...  
13    }  
14 }
```

```
1 class A {  
2     private void f(B b) {  
3         int total;  
4         total = b.getTotal();  
5         ...  
6     }  
7 }
```

RESUMINDO

Recomendação comum em projeto de software:

Maximize a coesão, minimize o acoplamento

RESUMINDO

Recomendação comum em projeto de software:

Maximize a coesão, minimize o acoplamento

Mas cuidado: minimize (ou elimine) principalmente o acoplamento ruim

VAMOS PENSAR...

- 1 Suponha que você ficou responsável pela implementação de um sistema que vai demandar ~100 KLOC.

Proponha hipoteticamente um design para sua implementação que:

- ▶ tenha a pior coesão possível
- ▶ mas, ao mesmo tempo, tenha o melhor acoplamento possível

VAMOS PENSAR...

- ② Você consegue dizer qual valor o seguinte programa vai imprimir (sem conhecer o código de f)?

```
1 int g = 1; // global
2 f();
3 printf("%d",g);
```

VAMOS PENSAR...

- ② Você consegue dizer qual valor o seguinte programa vai imprimir (sem conhecer o código de f)?

```
1 int g = 1; // global
2 f();
3 printf("%d",g);
```

Usando esse programa como exemplo, explique porque variáveis globais dão origem a designs ruins?

VAMOS PENSAR...

- ② Você consegue dizer qual valor o seguinte programa vai imprimir (sem conhecer o código de f)?

```
1 int g = 1; // global
2 f();
3 printf("%d",g);
```

Usando esse programa como exemplo, explique porque variáveis globais dão origem a designs ruins?

Sistemas com muitas variáveis globais não atendem a qual propriedade de projeto?

VAMOS PENSAR...

- 1 Seja o seguinte código que realiza operações em um conjunto de contas bancárias.
 - ▶ Qual a propriedade de projeto é violada por esse código
 - ▶ Como você melhoraria o design desse código?

```
1 var saldo = [150, 10, 90]; // global
2
3 function depositar(conta, valor) {
4     saldo[conta] += valor;
5 }
6
7 function getSaldo(conta) {
8     return saldo[conta];
9 }
```

VAMOS PENSAR...

④ Sejam duas classes A e B que:

- ▶ estão implementadas em diretórios diferentes
- ▶ a classe A possui uma referência no seu código para B

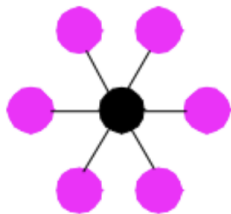
Então, sempre que um programador precisa, como parte de uma tarefa de manutenção, modificar classes A e B que atendem a tais critérios ele conclui a tarefa movendo B para o mesmo diretório de A.

Agindo dessa maneira, o programador estará melhorando qual propriedade de projeto (quando medida entre diretórios)?

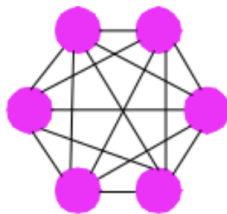
E qual propriedade é afetada de modo negativo?

VAMOS PENSAR...

- 5 Genericamente falando, qual dos seguintes projetos é melhor? Justifique (nodos = classes; arestas = dependências)



(A)

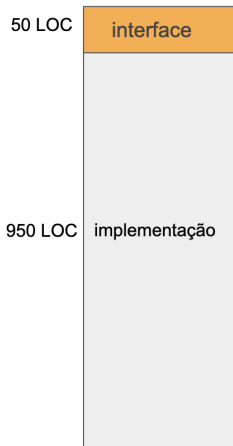


(B)

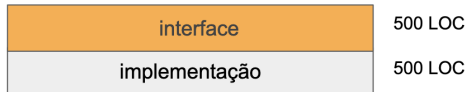
Fonte: Bertrand Meyer, Object-oriented software construction, 1997

VAMOS PENSAR...

6 Qual dos seguintes módulos é melhor? Justifique.



(A)



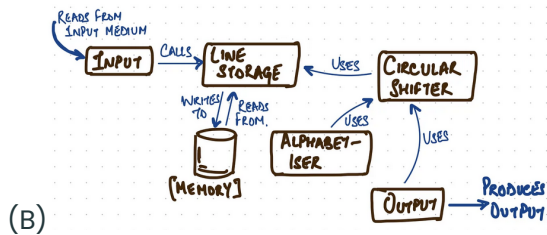
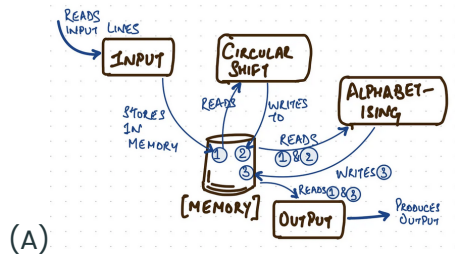
(B)

VAMOS PENSAR...

- 7 Seguem duas modularizações de um programa que lê um conjunto de linhas da entrada, cria todos os “shifts circulares” dessas linhas e depois imprime os shifts em ordem alfabética.

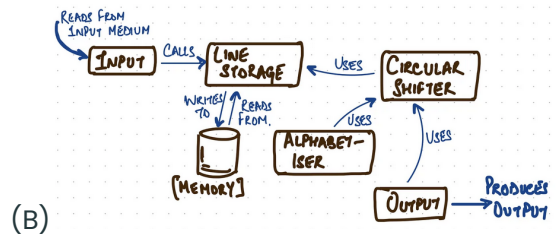
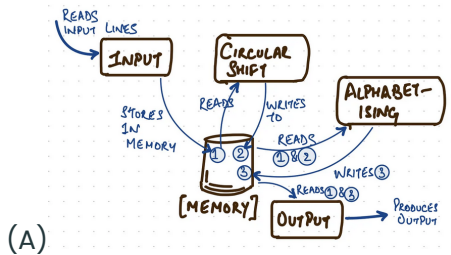
Qual das duas modularizações é melhor?

Qual propriedade de projeto ela atende (em detrimento da outra)?



CIRCULAR SHIFTS

- 1 software engineering is a discipline
- 2 engineering is a discipline software
- 3 is a discipline software engineering
- 4 a discipline software engineering is
- 5 discipline software engineering is a



VAMOS PENSAR...

- 8 Suponha dois métodos f e g . Existe um acoplamento temporal entre eles quando para chamar g temos que chamar antes f .

Dê um exemplo aceitável e bastante comum de acoplamento temporal (ou seja, dê nomes concretos de métodos f e g).

VAMOS PENSAR...

- ⑧ Suponha dois métodos *f* e *g*. Existe um acoplamento temporal entre eles quando para chamar *g* temos que chamar antes *f*.

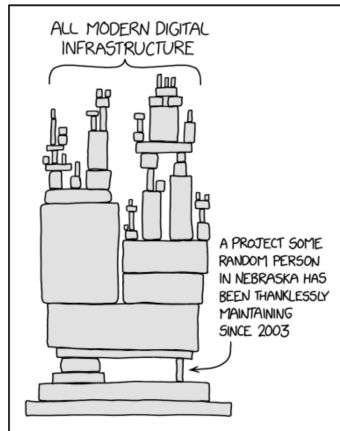
Dê um exemplo aceitável e bastante comum de acoplamento temporal (ou seja, dê nomes concretos de métodos *f* e *g*).

O acoplamento temporal do seguinte código é aceitável ou ruim? Se for ruim, sugira uma refatoração para melhorar o código.

```
1 var circle = new Circle();  
2 circle.setRadius(5);  
3 circle.getArea();
```

VAMOS PENSAR...

- 9 A seguinte figura, de certo modo, ilustra os problemas derivados de qual propriedade de projeto?



<https://xkcd.com/2347>

Princípios de Projeto Modular

Introdução

Propriedades de Projeto

Princípios de Projeto

PRINCÍPIOS DE PROJETO

Princípio de Projeto

Responsabilidade Única

Segregação de Interfaces

Inversão de Dependências

Prefira Composição a Herança

Demeter

Aberto/Fechado

Substituição de Liskov

Propriedade de Projeto

Coesão

Coesão

Acoplamento

Acoplamento

Ocultamento de Informação

Extensibilidade

Extensibilidade

PRINCÍPIOS DE PROJETO

Princípio de Projeto

Responsabilidade Única

Segregação de Interfaces

Inversão de Dependências

Prefira Composição a Herança

Demeter

Aberto/Fechado

Substituição de Liskov

Propriedade de Projeto

Coesão

Coesão

Acoplamento

Acoplamento

Ocultamento de Informação

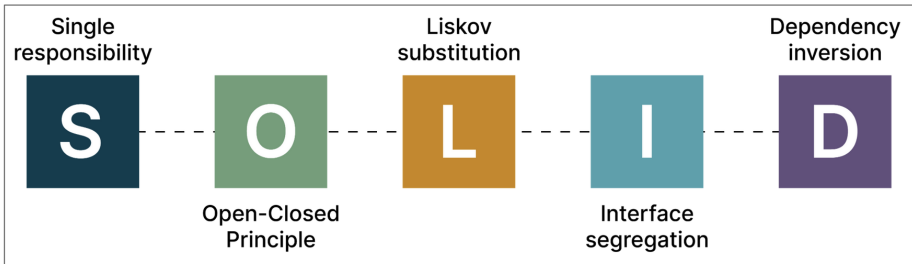
Extensibilidade

Extensibilidade

Princípio de Projeto: Recomendação mais explícita

Propriedade de Projeto: Consequência (o que vamos ganhar seguindo o princípio)

PRINCÍPIOS DE PROJETO



PRINCÍPIO DA RESPONSABILIDADE ÚNICA

Toda classe deve ter uma única responsabilidade

Deve existir um único motivo para modificar uma classe

PRINCÍPIO DA RESPONSABILIDADE ÚNICA

```
1 class Disciplina {  
2     int total;  
3     int desistentes;  
4  
5     //sets e gets...  
6  
7     void calculaIndiceDesistencia() {  
8         double indice = total/desistentes;  
9         System.out.println(indice);  
10    }  
11  
12 }
```

PRINCÍPIO DA RESPONSABILIDADE ÚNICA

```
1 class Disciplina {  
2     int total;  
3     int desistentes;  
4  
5     //sets e gets...  
6  
7     void calculaIndiceDesistencia() {  
8         double indice = total/desistentes;  
9         System.out.println(indice);  
10    }  
11  
12 }
```

Responsabilidade #1: calcular índice de desistência

Responsabilidade #2: imprimir índice de desistência

VERSÃO COM SEPARAÇÃO DE RESPONSABILIDADES

```
1 class Console {
2     void imprimeIndiceDesistencia(Disciplina disciplina) {
3         double indice = disciplina.calculaIndiceDesistencia();
4         System.out.println(indice);
5     }
6 }
7
8 class Disciplina {
9     int total;
10    int desistentes;
11    double calculaIndiceDesistencia() {
12        double indice = total/desistentes;
13        return indice;
14    }
15 }
```

VERSÃO COM SEPARAÇÃO DE RESPONSABILIDADES

```
1 class Console {  
2 ...  
3 }  
4  
5 class Disciplina {  
6 ...  
7 }
```

Classe de apresentação (interface com usuário)

- ▶ desenvolvedor frontend

Classe de negócio (regra de negócio ou classe de domínio)

- ▶ desenvolvedor backend
- ▶ mais fácil de ser testada (teste de unidade)

PRINCÍPIO DA SEGREGAÇÃO DE INTERFACES

Interfaces devem ser pequenas, coesas e específicas para cada tipo de cliente

Caso particular do princípio anterior, mas voltado para interfaces

PRINCÍPIO DA SEGREGAÇÃO DE INTERFACES

```
1 interface Funcionario {  
2  
3     double getSalario();  
4  
5     double getFGTS();// apenas funcionários CLT  
6  
7     int getSIAPE();// apenas funcionários públicos  
8  
9     ...  
10 }
```

VERSÃO QUE ATENDE SEGREGAÇÃO DE INTERFACES

```
1 interface Funcionario {
2     double getSalario();
3     ...
4 }
5
6 interface FuncionarioCLT extends Funcionario {
7     double getFGTS();
8     ...
9 }
10
11 interface FuncionarioPublico extends Funcionario {
12     int getSIAPE();
13     ...
14 }
```

PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIAS

Podemos chamar esse princípio de “Prefira Interfaces a Classes”

Pois transmite melhor a sua ideia!

PRINCÍPIO DA INVERSÃO DE DEPENDÊNCIAS

Podemos chamar esse princípio de “Prefira Interfaces a Classes”

Pois transmite melhor a sua ideia!

```
1 class ControleRemoto {  
2     TVsamsung tv;  
3     ...  
4 }  
5  
6 class TVsamsung {  
7     ...  
8 }
```

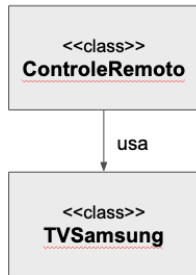
Qual o possível problema de design?

EXEMPLO USANDO INVERSÃO DE DEPENDÊNCIAS

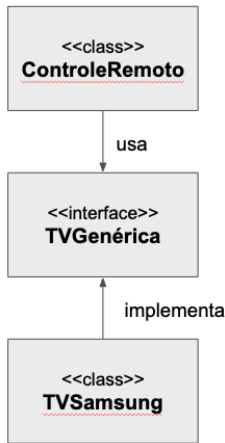
```
1 class ControleRemoto {  
2     TVGenerica tv;  
3     ...  
4 }  
5  
6 interface TVGenerica {  
7     ...  
8 }  
9  
10 class TVsamsung implements TVGenerica {  
11     ...  
12 }
```

EXEMPLO USANDO INVERSÃO DE DEPENDÊNCIAS

Sem DIP



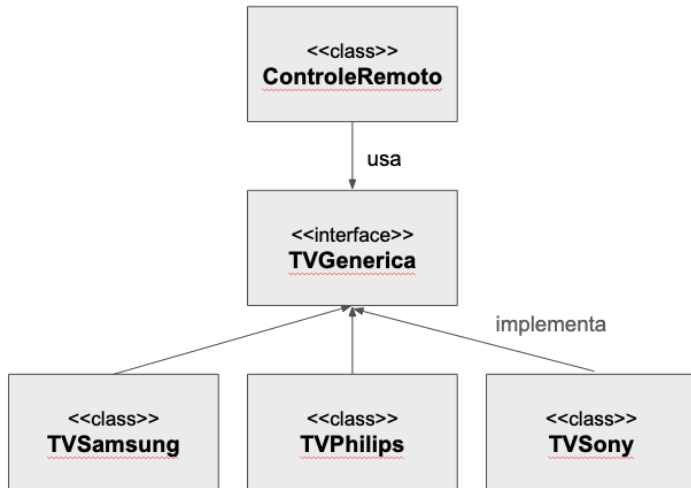
Com DIP



Vantagem: controle remoto genérico, para qualquer TV

Posso mudar de TV, sem alterar o código da classe ControleRemoto

DEPOIS DE UM TEMPO...



PREFIRA COMPOSIÇÃO A HERANÇA

Na década de 80, quando orientação a objetos tornou-se popular, as pessoas começaram a abusar de herança

Achavam que herança iria ser uma *bala de prata*, promover reúso em larga escala, etc.

PREFIRA COMPOSIÇÃO A HERANÇA

Herança: relação do tipo “é-um”

```
class MotorGasolina extends Motor {  
    ... // herda atributos e métodos de motor  
}
```

Composição: relação do tipo “possui”

```
class Painel {  
    ContaGiros cg;  
    ...  
}
```

PREFIRA COMPOSIÇÃO A HERANÇA

Prefira Composição a Herança

- ▶ **não force o uso de herança**

PRINCÍPIO DE DEMETER

Demeter: nome de um grupo de pesquisa americano

Evite longas cadeias de chamadas de métodos

Exemplo:

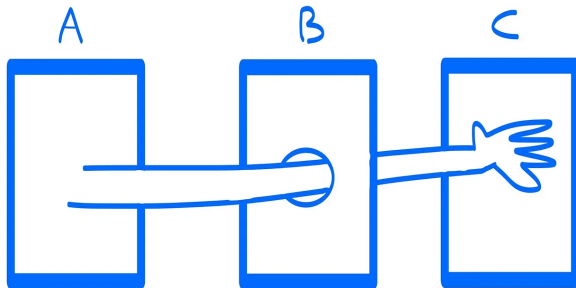
```
obj.getA().getB().getC().getD().getOqueEuPreciso();
```

Vários objetos de passagem

PRINCÍPIO DE DEMETER

Motivo:

- ▶ Longas cadeias de chamadas quebram encapsulamento
- ▶ Não quero passar por A, B, C,... até obter que eu preciso



<https://medium.com/@evan.hopkins.us/>

the-law-of-demeter-and-its-application-to-react-ab1e054f13c5

CIC0198 - TÉCNICAS DE PROGRAMAÇÃO 2 - Princípios de Projeto Modular

PRINCÍPIO DE DEMETER

```
1 class PrincipioDemeter {
2     T1 attr;
3
4     void f1() {
5         ...
6     }
7
8     void m1(T2 p) {    // método que segue Demeter
9         f1();          // caso 1: própria classe
10        p.f2();         // caso 2: parâmetro
11        new T3().f3();  // caso 3: criado pelo método
12        attr.f4();     // caso 4: atributo da classe
13    }
14
15    void m2(T4 p) {    // método que viola Demeter
16        p.getX().getY().getZ().doSomething();
17    }
18 }
```

MAS CUIDADO

Demeter e demais princípios são **recomendações**

Não devemos ser radicais e achar que cadeias de chamadas de métodos são totalmente proibidas

Casos isolados podem existir e terem uma boa justificativa

EXEMPLO JUSTIFICÁVEL DE CADEIAS DE MÉTODOS

```
1 const numbers = [1, 2, 3, 4, 5, 6];  
2 const result = numbers  
3   .map(num => num * 2)  
4   .filter(num => num % 3 === 0)  
5   .reduce((acc, num) => acc + num, 0);  
6 console.log(result); // Output: 18
```

Métodos da linguagem ou de sua API

PRINCÍPIO ABERTO/FECHADO

Proposto por Bertrand Meyer

Ideia: uma classe deve estar **fechada** para modificações, mas **aberta** para extensões



1988

EXPLICANDO MELHOR

Suponha que você vai implementar uma classe

EXPLICANDO MELHOR

Suponha que você vai implementar uma classe

Usuários ou clientes vão querer usar a classe (**óbvio!**)

EXPLICANDO MELHOR

Suponha que você vai implementar uma classe

Usuários ou clientes vão querer usar a classe (**óbvio!**)

Mas vão querer também customizar, parametrizar, configurar, flexibilizar e estender a classe!

EXPLICANDO MELHOR

Suponha que você vai implementar uma classe

Usuários ou clientes vão querer usar a classe (**óbvio!**)

Mas vão querer também customizar, parametrizar, configurar, flexibilizar e estender a classe!

Você deve se antecipar e tornar possível tais extensões

Mas sem que os clientes tenham que editar o código da classe

PRINCÍPIO ABERTO/FECHADO

Como tornar uma classe aberta a extensões, mas mantendo o seu código fechado para modificações?

PRINCÍPIO ABERTO/FECHADO

Como tornar uma classe aberta a extensões, mas mantendo o seu código fechado para modificações?

- ▶ Parâmetros
- ▶ Funções de mais alta ordem
- ▶ Padrões de projeto
- ▶ Herança
- ▶ etc

EXEMPLO

```
1 List<String> nomes;  
2 nomes = Arrays.asList("joao", "maria", "alexandre", "ze");  
3 Collections.sort(nomes);
```

Ordena a lista passada com parâmetro

```
1 System.out.println(nomes);  
2 // resultado: ["alexandre","joao","maria","ze"]
```

EXEMPLO

Mas agora eu quero ordenar as strings da lista pelo seu tamanho, isto é, pelo número de chars!

EXEMPLO

Mas agora eu quero ordenar as strings da lista pelo seu tamanho, isto é, pelo número de chars!

Será que o método sort está aberto (preparado) para permitir essa extensão?

EXEMPLO

Mas agora eu quero ordenar as strings da lista pelo seu tamanho, isto é, pelo número de chars!

Será que o método `sort` está aberto (preparado) para permitir essa extensão?

Mas, mantendo o seu código fechado, isto é, sem ter que mexer no seu código.

SOLUÇÃO

```
1 Comparator<String> comparador = new Comparator<String>() {  
2     public int compare(String s1, String s2) {  
3         return s1.length() - s2.length();  
4     }  
5 };  
6 Collections.sort(nomes, comparador);  
7  
8 System.out.println(nomes);  
9 // resultado: [ze, joao, maria, alexandre]
```

SOLUÇÃO

```
1 Comparator<String> comparador = new Comparator<String>() {  
2     public int compare(String s1, String s2) {  
3         return s1.length() - s2.length();  
4     }  
5 };  
6 Collections.sort(nomes, comparador);  
7  
8 System.out.println(nomes);  
9 // resultado: [ze, joao, maria, alexandre]
```

Não existe almoço grátis: cliente (código que chama “sort”) teve que implementar e passar como parâmetro uma função que define o critério a ser usado na ordenação

PRINCÍPIO ABERTO/FECHADO

Resumindo: ao implementar uma classe, pense em pontos de extensão!

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV

Nome é uma referência à Profa. Barbara Liskov

Princípio define boas práticas para uso de herança

Especificamente, boas práticas para redefinição de métodos em subclasses



PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV

Primeiro: vamos entender o termo “substituição”

```
1 void f(A a) {  
2     ...  
3     a.g();  
4     ...  
5 }  
6  
7 f(new B1()); // f pode receber objetos da subclasse B1  
8 ...  
9 f(new B2()); // e de qualquer subclasse de A, como B2  
10 ...  
11 f(new B3()); // e B3
```

PRINCÍPIO DE SUBSTITUIÇÃO DE LISKOV

Tipo A pode ser substituído por B1, B2, B3,...

Desde que eles sejam subclasses de A

Substituições de A por B podem acontecer desde que B ofereça os mesmos serviços de A

Ou seja: para um código feito para usar A, a substituição de A por B é “imperceptível”

EXEMPLO

Exemplo que segue Substituição de Liskov

```
1 class ControleRemoto {  
2     // alcance de 10 m  
3 }  
4 class ControleRemotoPremium extends ControleRemoto {  
5     // alcance de 20 m  
6 }
```

Exemplo que **não** segue Substituição de Liskov

```
1 class ControleRemoto {  
2     // alcance de 10 m  
3 }  
4 class ControleRemotoXYZ extends ControleRemoto {  
5     // alcance de 5 m  
6 }
```

EXERCÍCIOS SOBRE PRINCÍPIOS DE PROJETO

- 1 Qual princípio de projeto é violado por uma chamada como a mostrada abaixo?

Qual mudança de projeto você faria na classe Biblioteca (tipo de bib) para remover essa violação?

```
1 bib.getAcervo().getAreaConhecimento("ES").  
2   getLivros().find("ESM").getNumExemplares();
```

EXERCÍCIOS SOBRE PRINCÍPIOS DE PROJETO

② Suponha a seguinte classe:

```
1 class Tabela {
2     ...
3     void imprime() {
4         // código que imprime cabeçalho
5         // código que imprime cada linha da tabela
6         // código que imprime rodapé
7     }
8     ...
9 }
```

Esse código não segue o Princípio Aberto/Fechado, pois no nosso sistema é importante poder variar as mensagens de cabeçalho e rodapé.

Como você mudaria a implementação dessa classe para atender ao Princípio Aberto/Fechado?

EXERCÍCIOS SOBRE PRINCÍPIOS DE PROJETO

- ③ Suponha uma classe Calculadora com um método que verifica se um número entre 0 e 10000 é primo.

Suponha ainda que um algoritmo mais eficiente para essa tarefa foi implementado em uma subclasse CalculadoraRapida.

No entanto, ele somente é capaz de trabalhar com números entre 1000 e 9000.

```
1 class Calculadora {  
2     boolean isPrimo(n) {  
3         // 0 <= n < 10000  
4     }  
5 }
```

```
1 class CalculadoraRapida extends Calculadora {  
2     boolean isPrimo(n) {  
3         // 1000 <= n < 9000  
4     }  
5 }
```

Qual princípio de projeto é violado nessa implementação? Justifique.

EXERCÍCIOS SOBRE PRINCÍPIOS DE PROJETO

- ④ Suponha que você concluiu um curso de extensão oferecido pelo CIC e quer receber seu certificado. Para isso, você teve que:
- ▶ Mandar uma msg para o coordenador do curso, que te pediu para mandar uma msg para secretaria do CIC.
 - ▶ Então, você mandou uma msg para a secretaria do CIC, que te pediu para mandar uma mensagem para o IE.
 - ▶ Então, você mandou uma msg para o IE, que te pediu para mandar uma msg para a DEX.
 - ▶ Então, você mandou uma msg para a DEX, que finalmente retornou o seu certificado.

Essa sequência de passos ilustra uma violação de qual princípio de projeto? Além de ter que mandar várias mensagens, qual um outro problema dessa “solução”?

EXERCÍCIOS SOBRE PRINCÍPIOS DE PROJETO

- 5) Suponha que você está implementando um sistema S e precisa usar um serviço de processamento de pagamentos X para, por exemplo, aceitar pagamentos via cartões de crédito. No entanto, você não quer acoplar S a X, pois futuramente você pode optar por um outro serviço Y.
- a) Explique como você faria para desacoplar S de X.
 - b) Qual princípio de projeto você usou para garantir esse desacoplamento?

EXERCÍCIOS SOBRE PRINCÍPIOS DE PROJETO

- ⑥ Em Engenharia de Software, às vezes usamos soluções mais complexas em contextos nos quais isso não é necessário. Esse problema é chamado de *overengineering* ou otimização prematura.

Então crie e descreva um contexto no qual o uso de um dos princípios de projeto que estudamos é uma otimização prematura.

Se quiser, você pode criar e explicar esse contexto usando o mesmo exemplo que usamos na explicação do princípio.

Este documento está licenciado sob a Licença Atribuição-NãoComercial-Compartilhalgual 4.0 Internacional Creative Commons. Para visualizar uma cópia desta licença, visite <http://creativecommons.org/licenses/by-nc-sa/4.0/> ou mande uma carta para Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

